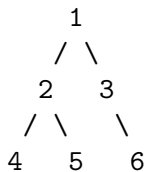


Part 3: BFS and Level Order Traversal

Processing a tree one level at a time

The problem DFS cannot solve comfortably

Look at our tree again.



Suppose the question is: “print the nodes level by level” or “what is the rightmost node on each level” or “which level has the largest sum”. DFS jumps from 4 straight to 5 and then all the way over to 3. Nodes on the same level are visited far apart in time. You can still do it with DFS by tracking depth, and sometimes we do, but the natural tool here is BFS.

BFS finishes an entire level before moving down. On our tree it produces:

```
level 0:  1
level 1:  2 3
level 2:  4 5 6
```

Queue intuition

BFS needs a **queue**: first in, first out. Think of a line at a ticket counter. The person who joined first is served first, and new arrivals go to the back.

```
queue<TreeNode*> q;
q.push(node);      // join the back of the line
q.front();          // who is at the front
q.pop();            // remove the front person
q.empty();          // is the line empty
q.size();           // how many are waiting
```

The algorithm in one sentence: **take the node at the front, note its value, and put its children at the back.**

Why does that produce level order? Because a node’s children always join the queue after every node of the current level has already joined. The queue never lets a deeper node overtake a shallower one.

If you want a mental image, the queue at any moment holds the “frontier”: the last part of one level plus the first part of the next.

Basic level order, everything in one list

```
vector<int> levelOrder(TreeNode* root) {
    vector<int> out;
    if (root == NULL) return out;

    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        TreeNode* node = q.front();
        q.pop();

        out.push_back(node->val);

        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
    return out;
}
```

Dry run

start	q = [1]		out = []
pop 1	push 2, 3	q = [2,3]	out = [1]
pop 2	push 4, 5	q = [3,4,5]	out = [1,2]
pop 3	push 6	q = [4,5,6]	out = [1,2,3]
pop 4	no children	q = [5,6]	out = [1,2,3,4]
pop 5	no children	q = [6]	out = [1,2,3,4,5]
pop 6	no children	q = []	out = [1,2,3,4,5,6]
queue empty, stop			

Every node enters the queue exactly once and leaves exactly once.

Time $O(N)$. Space $O(W)$, where W is the maximum width of the tree. In the worst case the last level of a perfect tree holds about $N/2$ nodes, so this is $O(N)$. Note the difference from DFS, whose space is $O(H)$. For a wide shallow tree DFS is cheaper, for a deep skewed tree BFS is cheaper.

Level order as a vector of vectors

This is LeetCode 102, and it is the version you will actually reuse. The output for our tree should be `[[1], [2,3], [4,5,6]]`.

The only new idea is one line: **before the inner loop starts, record how many nodes are currently in the queue. That count is exactly the size of the current level.**

```

vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (root == NULL) return res;

    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        int sz = q.size();           // size of the current level, snapshot it
        vector<int> level;

        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front();
            q.pop();

            level.push_back(node->val);

            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}

```

Why `int sz = q.size()` must come before the loop

During the inner loop you are pushing children, so `q.size()` keeps changing. If you write `for (int i = 0; i < q.size(); i++)` the condition is re-evaluated every iteration and you will happily consume the next level too, producing one giant level. Take the snapshot once. This is the single most common BFS bug.

Dry run

```

outer pass 1:  sz = 1,  q = [1]
               pop 1, level = [1], push 2,3
               res = [[1]]                                q = [2,3]

outer pass 2:  sz = 2,  q = [2,3]
               pop 2, level = [2], push 4,5
               pop 3, level = [2,3], push 6
               res = [[1],[2,3]]                            q = [4,5,6]

outer pass 3:  sz = 3,  q = [4,5,6]
               pop 4, pop 5, pop 6, no children
               res = [[1],[2,3],[4,5,6]]                    q = []

```

This template is the skeleton for every level based problem. Learn it once and only change what happens inside the inner loop.

Variations built on the same template

Level sums

```
vector<int> levelSums(TreeNode* root) {
    vector<int> res;
    if (root == NULL) return res;
    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        int sz = q.size();
        long long sum = 0;
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            sum += node->val;
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back((int)sum);
    }
    return res;
}
```

For our tree: [1, 5, 15]. Use long long when values can be large and a level can be wide. LeetCode 1161 (maximum level sum) is this function plus an argmax.

Level maximum

Replace the accumulator:

```
int mx = INT_MIN;
// inside the inner loop
mx = max(mx, node->val);
```

For our tree: [1, 3, 6]. That is LeetCode 515.

Level averages

Same loop, keep sum and divide by sz. Use double, and accumulate in long long or double to avoid overflow. LeetCode 637.

Right side view

The right side view is what you see standing to the right of the tree: the **last** node of every level.

```
vector<int> rightSideView(TreeNode* root) {
    vector<int> res;
    if (root == NULL) return res;
    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
```

```

    int sz = q.size();
    for (int i = 0; i < sz; i++) {
        TreeNode* node = q.front(); q.pop();
        if (i == sz - 1) res.push_back(node->val);    // last of this level
        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
}
return res;
}

```

For our tree: 1 3 6. The left view is the same code with `i == 0`. For our tree: 1 2 4.

Note carefully: the right view is **not** “keep going right from the root”. On our tree that path is 1, 3, and then 3 has no right child, so you would miss 6 entirely. The right view is the last node of each level, wherever it comes from.

Zigzag level order

Left to right on even levels, right to left on odd levels (LeetCode 103). Do not use a stack and do not reverse the queue. Build the level normally, then reverse the level vector when needed:

```

bool leftToRight = true;
// after building the level vector:
if (!leftToRight) reverse(level.begin(), level.end());
res.push_back(level);
leftToRight = !leftToRight;

```

Bottom-up level order

LeetCode 107 wants levels from the bottom up. Build normally, then `reverse(res.begin(), res.end())`. Do not try to be clever.

Minimum depth

The depth of the nearest leaf. BFS is genuinely better here: the first leaf you meet is the answer, and you can stop immediately.

```

int minDepth(TreeNode* root) {
    if (root == NULL) return 0;
    queue<TreeNode*> q;
    q.push(root);
    int depth = 1;

    while (!q.empty()) {
        int sz = q.size();
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            if (node->left == NULL && node->right == NULL) return depth;
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        depth++;
    }
}

```

```
    return depth;
}
```

On a tree with a million nodes where one leaf sits at depth 2, DFS explores everything and BFS stops almost immediately. When a problem says “nearest”, “minimum number of steps”, or “first level where something happens”, think BFS.

BFS or DFS, how to decide

Situation	Use
Answer depends on levels or distance from the root	BFS
Nearest or minimum depth style question	BFS
Right view, left view, level sums, zigzag	BFS
Answer is built from subtree answers	DFS, postorder
Path from root to some node	DFS, preorder
Height, diameter, balanced, path sums	DFS
Tree is very deep and narrow, recursion may overflow	BFS, or iterative DFS
Tree is very wide and shallow, memory matters	DFS

Almost anything BFS can do, DFS can also do by carrying the depth as a parameter. The right view, for instance, can be done with a “root, right, left” DFS that records the first node it meets at each new depth. Use whichever you can write correctly under pressure. For genuinely level based problems, BFS is usually shorter and easier to explain.

Common mistakes

- **Recomputing `q.size()` inside the loop condition.** Snapshot it into `sz` first.
- **Pushing NULL children.** Then `node->val` crashes later. Either guard on push (preferred) or guard on pop, but do not do neither.
- **Forgetting the empty tree.** `if (root == NULL) return {};` on the first line.
- **Using vector with `erase(begin())` as a queue.** That is $O(N)$ per removal and turns an $O(N)$ algorithm into $O(N^2)$. Use `queue<TreeNode*>`.
- **Calling `q.front()` after `q.pop()`.** Read the value first, then pop.
- **Trying to detect the end of a level by comparing values.** Values repeat. Use the size snapshot.
- **Thinking the right view means always going right.** It does not.

Check yourself

1. Why does taking a snapshot of the queue size cleanly separate the levels?
2. Our tree gives `[[1], [2,3], [4,5,6]]`. What would BFS give for a completely skewed tree of 4 nodes?

3. What is the worst case space of BFS, and which tree shape causes it?
4. Which is better for minimum depth, BFS or DFS, and why?
5. You need the leftmost node of the deepest level. Sketch a BFS that finds it with almost no extra code.
6. Can you produce a right side view with DFS? What visiting order would you use?